ATIVIDADE – Sistema de Gestão de Livros

Spring Boot Web – Persistência em Memória

Objetivo

Implementar uma API REST utilizando Spring Boot, mantendo a arquitetura em camadas:

Entity → Repository → Service → Controller

Sem uso de JPA, banco de dados ou frameworks adicionais.

PARTE 1 — Criação do Projeto

1. Acessar o Spring Initializr

Via:

- https://start.spring.io

  ou

- Spring Initializr da IDE

Configurações:

Campo	Valor
Project	Maven
Language	Java
Spring Boot	versão estável
Group	br.com.seunome
Artifact	biblioteca
Packaging	Jar
Java	17 ou 21

Dependências:

- Spring Web
- Spring DevTools (opcional)

Não adicionar:

- JPA
- Banco de dados
- Lombok

---

2. Importar na IDE

- Baixar o ZIP
- Importar como projeto Maven
- Executar a aplicação
- Verificar se sobe corretamente em http://localhost:8080

Se a aplicação iniciar sem erros, prossiga.

PARTE 2 — Estrutura de Pacotes

Criar manualmente a estrutura:

br.com.seunome.biblioteca

├── controller

├── service

├── repository

└── domain

Regras obrigatórias:

- Controller não acessa Repository diretamente
- Toda regra de negócio deve estar no Service
- Repository não contém regra de negócio

PARTE 3 — Entidade (Domain)

Arquivo: Livro.java

Pacote: domain

Atributos obrigatórios:

- id (auto incremento manual)
- titulo
- autor
- isbn
- anoPublicacao
- disponivel (true por padrão)

Implementar:

1. Construtor com validações
2. Getters
3. Setters com validações
4. Métodos:
    - marcarComoDisponivel()
    - marcarComoIndisponivel()

Validações obrigatórias:

Título:

- Não nulo
- Mínimo 3 caracteres após trim

Autor:

- Mesmo padrão do título

ISBN:

- Formato: XXX-X-XXXX-XXXX-X
- Validar com regex
- Unicidade será validada no Service

Ano:

- Entre 1500 e 2025

Checklist:

- ID com contador estático
- Validações funcionando
- Testado criando objetos manualmente

PARTE 4 — Repository

1. Criar Interface

Pacote: repository

Métodos obrigatórios:

- salvar
- listarTodos
- buscarPorId
- buscarPorIsbn
- atualizar
- deletar
- contar (opcional)

Utilizar Optional onde necessário.

---

2. Implementação em Memória

Classe: LivroRepositoryMemoria

Requisitos:

- Implementar interface
- Utilizar Map<Integer, Livro>
- Anotar com @Repository

Regras:

- Não aceitar livro nulo
- Retornar nova lista ao listar
- Buscar por ISBN utilizando stream

Checklist:

- CRUD funcionando via Map
- Nenhuma regra de negócio aqui
- Testado com dados fixos

PARTE 5 — Service

Pacote: service

Anotar classe com @Service

Injeção via construtor

Métodos obrigatórios:

- cadastrarLivro()
- listarLivros()
- buscarLivroPorId()
- atualizarLivro()
- removerLivro()
- marcarComoIndisponivel()
- marcarComoDisponivel()

Regras obrigatórias:

Cadastro:

- Validar ISBN único
- Delegar persistência ao repository

Atualização:

- Não permitir alterar ISBN
- Validar existência antes

Remoção:

- Lançar exceção se não existir

Checklist:

- Nenhuma regra no Controller
- Nenhuma regra no Repository
- Service centraliza regras

PARTE 6 — Controller REST

Pacote: controller

Anotações obrigatórias:

- @RestController
- @RequestMapping("/livros")

Endpoints obrigatórios:

```
POST /livros
GET /livros
GET /livros/{id}
PUT /livros/{id}
PATCH /livros/{id}/indisponivel
PATCH /livros/{id}/disponivel
DELETE /livros/{id}
```

Regras:

- Usar @RequestBody
- Usar @PathVariable
- Retornar ResponseEntity quando necessário
- Não implementar regra de negócio

Controller deve apenas:

- Receber requisição
- Chamar o Service
- Retornar resposta

Checklist:

- Testado com Postman ou Insomnia
- Status HTTP corretos (200, 201, 204, 404)
- JSON estruturado corretamente

Boas Práticas Obrigatórias

- Injeção por construtor
- Separação clara de camadas
- Não usar static no Service
- Não acessar Map fora do Repository
- Não usar JPA
- Não usar banco de dados
- Não usar Lombok

Entrega

Projeto Spring Boot completo contendo:

- Estrutura organizada
- Camadas separadas
- Regras no Service
- Persistência em memória
- API funcionando

Nome do arquivo:

SistemaLivrosWeb_SeuNome_SuaMatricula.zip